Dimensionality reduction and unsupervised learning

Olivetti faces · four hundred photographs, no labels

LECTURE 8 | GÉRON CH. 7 & 8

Applications of Machine Learning

BSc Mathematics of Artificial Intelligence · Third year

Where we are

Seven lectures, and every one of them was handed the answers:

Lecture	What the data came with
1–2 · California housing	a price for every district
3 · MNIST	a digit for every image
4–5 · Titanic	a survivor flag for every passenger
6–7 · CoverType	a cover type for every plot
8 · Olivetti faces	**✗ nothing**

Today there is no target column. That changes what a metric can be, what a baseline is, and how you find out you were wrong.

Today

1. Four hundred points in four thousand dimensions, and what that costs (10 min)
2. **The mathematics** — PCA as the SVD of the centred data, and the Johnson–Lindenstrauss bound (20 min)
3. **Reducing the dimension** — four methods, timed and scored (25 min)
4. **Clustering** — k-means, and how to choose k (20 min)
5. Beyond k-means: density, mixtures, anomalies, forty labels (15 min)

Chapters 7 and 8 together, because the second one does not work on this data until the first one has been applied to it.

FIRST TEN MINUTES

Four hundred points in four thousand dimensions

10 minutes

The brief

> **THE REQUEST**
>
> “We have a photographic archive. We do not know how many distinct people are in it. Group the photographs by person, so that a human can name a *group* instead of naming every photograph.”

- The archive is not annotated, and nobody remembers who is in it
- A trained annotator costs money and time per photograph
- We are funded for **forty** labels. That is all

This is the ordinary situation outside a benchmark: data is cheap, labels are not.

Why this is a different kind of problem

Supervised

- You are given $(\mathbf{x}^{(i)}, y^{(i)})$
- The metric compares prediction to truth
- The baseline is “predict the majority”
- You know when you are wrong

Unsupervised

- You are given $\mathbf{x}^{(i)}$ only
- The metric compares the grouping to *itself*
- The baseline is “group at random”
- You find out you were wrong when someone looks

THE CONSEQUENCE FOR THE REST OF THE HOUR

Every number computed today is a statement about **geometry**, not about people. Whether the geometry *is* the people is a separate question, and it is answered by looking.

Where this sits in the book

Task	What it produces	Chapter
Dimensionality reduction	a shorter description of the same instance	7
Clustering	a group for every instance	8
Density estimation	a probability density over the space	8
Anomaly detection	a score for how unusual an instance is	8

All four appear today, in that order, and the order is not arbitrary: three of them are unusable on this data until the first has been applied.

Clustering is also used for customer segmentation, search-result grouping, image segmentation, and — as here — as a preprocessing step for a supervised model that cannot afford labels.

The corpus, and a photograph as a point

Quantity	Value
Photographs	400
Distinct people	40
Photographs per person	10
Pixels per photograph	4,096
The whole corpus in memory	6.6 MB

$$\mathbf{x}^{(i)} \in \mathbb{R}^{4096}, \qquad x_j \in [0$$

Flatten the 64 × 64 grid and a photograph is a vector. Every method today measures a distance between two of them.

✗ **Ten times more dimensions than points. Four hundred points cannot fill four thousand dimensions, and the first block of this lecture is about what follows from that.**

Forty people

One photograph of each of the 40 people. The corpus holds 400, and none of them arrives labelled

Same crop, same lighting rig, same background. What varies between rows is the person — and, as much, the lamp.

What “the same person” looks like

Ten photographs of one person, then ten of another: glasses on and off, lighting, expression, head angle

Glasses on and off, eyes shut, head turned, lit from either side. All twenty of these must end up in exactly two groups.

`y` exists. The brief does not have it

Olivetti is a benchmark, so it ships with the identity of every photograph. The archive we are pretending to hold does not.

> **THE RULE, KEPT FOR THE WHOLE LECTURE**
>
> `y` is used in exactly two ways, both announced on the slide where they happen: as the *forty labels we paid for*, and as an **audit**, to find out what those forty could not have told us.

Any other use of `y` is the same error as scoring on the test set, wearing different clothes. Slides marked *audit* are results the project itself could not have computed.

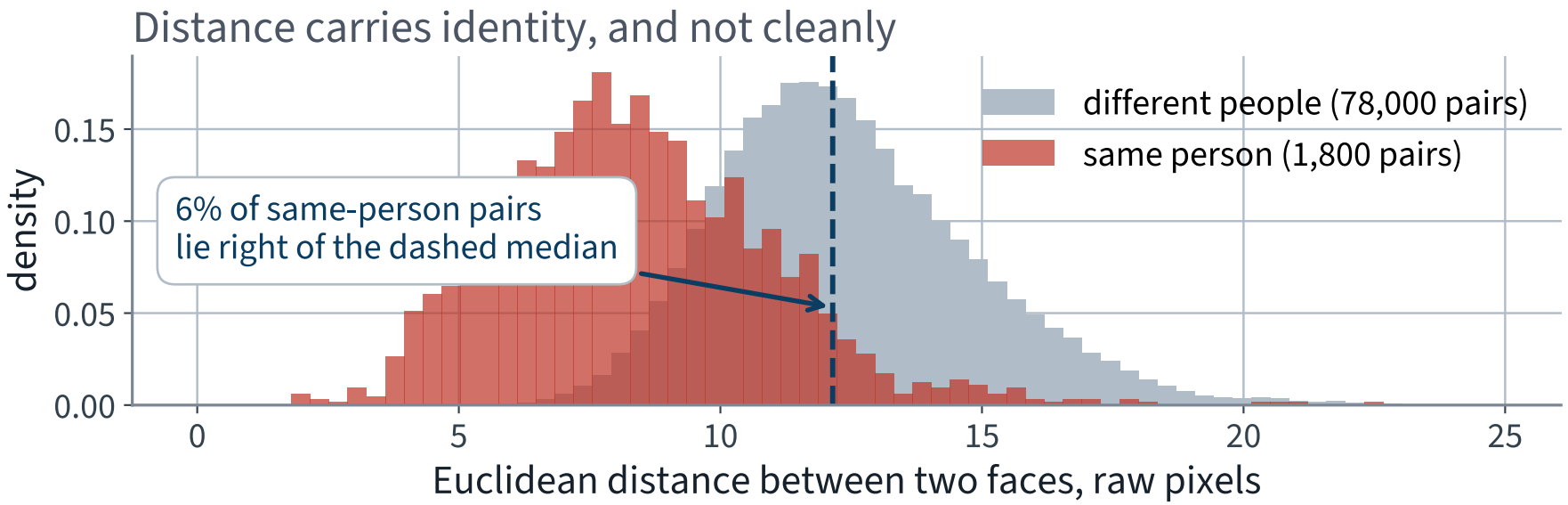
Before anything is fitted: can this work at all?

k-means groups points that are close in Euclidean distance. So one measurement decides whether the hour is worth spending (an audit — it uses the hidden labels):

Are two photographs of the same person closer than two photographs of different people?

Euclidean distance, raw pixels	Pairs	Mean	Median
Same person	1,800	8.38	8.15
Different people	78,000	12.41	12.15

The gap is real, and it is small



✗ 6% of same-person pairs sit further apart than the *median* different-person pair. Pixel distance is partly identity and partly lighting.

What that predicts, before we build anything

A FAILURE CONDITION, STATED IN ADVANCE

No method that groups by Euclidean distance in **raw pixels** will recover the forty people cleanly. Not because the method is bad — because the representation puts a lamp and a nose on the same axis, with the same weight.

Moving the light source changes hundreds of pixel values by a large amount. Changing the person changes a few dozen by a small one. Euclidean distance adds them all up equally.

✓ Measuring this costs one cell and tells you whether you are about to tune or about to hope.

And it is not free to work there

One model-selection sweep — k-means at every k from 2 to 60, ten restarts each, scored by silhouette:

Step	Seconds	Cost model
590 runs of Lloyd's algorithm	77	$O(\text{iterations} \times m\,k\,n)$
59 silhouette scores	2	$O(m^2 n)$
One sweep, total	✗ 78	two threads, one machine

Both cost models are **linear in n**, the number of features. Two of the three factors are properties of the problem. The third is a property of the *representation*.

✓ So: how many of those 4,096 numbers are doing work — and what exactly do you lose by throwing the rest away?

THE MATHEMATICS

PCA via the SVD, and Johnson–Lindenstrauss

20 minutes · examinable

Two questions, two answers

1. Among all d-dimensional subspaces, which one loses the least when you project onto it — and *how much* does it lose?
2. If all you need is that **distances** survive, how small can d be?

> **WHY YOU NEED BOTH**
>
> The first is a theorem about the optimal subspace and it names the price exactly. The second is a bound whose answer does not mention the dimension you started in — which is the fact worth carrying out of this lecture.

Notation, fixed for the whole derivation

- $\mathbf{X} \in \mathbb{R}^{m \times n}$ — one photograph per *row*: $m = 400$ rows, $n = 4096$ columns
- $\bar{\mathbf{x}}$ — the column means; $\tilde{\mathbf{X}} = \mathbf{X} - \mathbf{1}\bar{\mathbf{x}}^{\mathsf{T}}$ the centred data
- $\mathbf{W} = [\mathbf{w}_1 \cdots \mathbf{w}_d] \in \mathbb{R}^{n \times d}$ with $\mathbf{W}^{\mathsf{T}}\mathbf{W} = \mathbf{I}_d$ — an orthonormal basis of the subspace we are choosing
- $\mathbf{P} = \mathbf{W}\mathbf{W}^{\mathsf{T}}$ — the orthogonal projector onto it
- $\|\mathbf{A}\|_F^2 = \sum_{ij} a_{ij}^2 = \operatorname{tr}(\mathbf{A}^{\mathsf{T}}\mathbf{A})$

Everything below is linear algebra on $\tilde{\mathbf{X}}$. There is no probability in this derivation and none is needed.

Centre first, and say why

$$\tilde{\mathbf{X}} = \mathbf{X} - \mathbf{1}\bar{\mathbf{x}}^{\mathsf{T}}$$

Without centring, the direction of largest $\sum_i \left\| \mathbf{w}^{\mathsf{T}} \mathbf{x}^{(i)} \right\|^2$ is essentially $\bar{\mathbf{x}}$ itself, and the first component describes nothing but “this is a photograph of a face”.

WHEN THIS BITES

Scikit-Learn’s `PCA` centres for you. If you write the SVD by hand it does not, and the result is a plausible-looking first component and no error message. It is the commonest silent bug in a hand-rolled PCA.

Step 1 · what we are minimising

Project each centred face onto the subspace and measure how far it had to move:

$$E(\mathbf{W}) \;=\; \sum_{i=1}^{m} \left\| \tilde{\mathbf{x}}^{(i)} - \mathbf{W}\mathbf{W}^{\intercal}\tilde{\mathbf{x}}^{(i)} \right\|_2^2 \;=\; \left\| \tilde{\mathbf{X}} - \tilde{\mathbf{X}}\mathbf{W}\mathbf{W}^{\intercal} \right\|_F^2$$

The sum of squared distances from each point to the subspace. Minimise it over all $\mathbf{W}$ with $\mathbf{W}^{\intercal}\mathbf{W} = \mathbf{I}_d$.

Note what this is not: it is not a regression, there is no y, and nothing here is being predicted.

Step 2 · Pythagoras splits it in two

$\mathbf{P} = \mathbf{W}\mathbf{W}^\mathsf{T}$ is an orthogonal projector, so kept and lost are orthogonal:

$$\|\tilde{\mathbf{x}}\|^2 \;=\; \|\mathbf{W}^\mathsf{T}\tilde{\mathbf{x}}\|^2 \;+\; \|\tilde{\mathbf{x}} - \mathbf{P}\tilde{\mathbf{x}}\|^2$$

Summing over the m faces, and noting that $\|\tilde{\mathbf{X}}\|_F^2$ does not depend on $\mathbf{W}$:

$$E(\mathbf{W}) \;=\; \|\tilde{\mathbf{X}}\|_F^2 \;-\; \|\tilde{\mathbf{X}}\mathbf{W}\|_F^2$$

✓ **Minimising the error is maximising the projected variance. They are the same problem, not two related ones.**

Step 3 · write the surviving part as a trace

$$\|\tilde{\mathbf{X}}\mathbf{W}\|_F^2 \;=\; \operatorname{tr}\!\left(\mathbf{W}^\intercal\,\tilde{\mathbf{X}}^\intercal\tilde{\mathbf{X}}\,\mathbf{W}\right) \;=\; \sum_{j=1}^{d}\mathbf{w}_j^\intercal\,\mathbf{C}\,\mathbf{w}_j, \qquad \mathbf{C} = \tilde{\mathbf{X}}$$

$\mathbf{C}$ is $n \times n$, symmetric and positive semi-definite; it is $m-1$ times the sample covariance matrix.

So the whole problem is: **choose d orthonormal directions maximising $\operatorname{tr}(\mathbf{W}^\intercal\mathbf{C}\mathbf{W})$.**

Every remaining step is about that one quantity.

Step 4 · the singular value decomposition diagonalises $\mathbf{C}$

$$\tilde{\mathbf{X}} = \mathbf{U}\boldsymbol{\Sigma}\mathbf{V}^{\mathsf{T}}, \qquad \sigma_1 \geq \sigma_2 \geq \cdots \geq 0$$

$\mathbf{U}$ and $\mathbf{V}$ orthogonal, $\boldsymbol{\Sigma}$ diagonal. Every real matrix has one, which is why this works on any dataset rather than on well-behaved ones. Then

$$\mathbf{C} = \tilde{\mathbf{X}}^{\mathsf{T}}\tilde{\mathbf{X}} = \mathbf{V}\boldsymbol{\Sigma}^{\mathsf{T}}\mathbf{U}^{\mathsf{T}}\mathbf{U}\boldsymbol{\Sigma}\mathbf{V}^{\mathsf{T}} = \mathbf{V}\,\boldsymbol{\Sigma}^{2}\,\mathbf{V}^{\mathsf{T}}$$

The columns of $\mathbf{V}$ are the eigenvectors of $\mathbf{C}$, with eigenvalues σ_j^2. They are the **principal components**.

$\mathbf{C}$ is also the matrix Lecture 2's normal equation had to invert, and this decomposition says exactly when it cannot be: when some $\sigma_j = 0$.

Step 5 · rotate the unknown into the $\mathbf{V}$ basis

$\mathbf{V}$ is an orthogonal basis of $\mathbb{R}^n$, so any $\mathbf{W}$ can be written $\mathbf{W} = \mathbf{V}\mathbf{A}$ with $\mathbf{A}^\intercal\mathbf{A} = \mathbf{I}_d$. Then

$$\operatorname{tr}(\mathbf{W}^\intercal\mathbf{C}\mathbf{W}) \;=\; \operatorname{tr}(\mathbf{A}^\intercal\boldsymbol{\Sigma}^2\mathbf{A}) \;=\; \sum_{j=1}^{n} \sigma_j^2\, h_j$$

where h_j is the squared norm of row j of $\mathbf{A}$. Orthonormal columns constrain those weights to $0 \le h_j \le 1$ with $\sum_j h_j = d$. The problem has become a weighted sum with exactly d units of weight to spend.

Step 6 · a weighted sum with d units of weight to spend

Maximise $\sum_j \sigma_j^2 h_j$ subject to $0 \le h_j \le 1$ and $\sum_j h_j = d$. The σ_j^2 are sorted, so the answer is immediate:

$$\max_{\mathbf{W}^\intercal \mathbf{W} = \mathbf{I}_d} \operatorname{tr}(\mathbf{W}^\intercal \mathbf{C} \mathbf{W}) = \sum_{j=1}^{d} \sigma_j^2$$

Put a full unit of weight on each of the d largest σ_j^2 and nothing on the rest — that is $\mathbf{A} = \begin{bsmallmatrix}\mathbf{I}_d\\ \mathbf{0}\end{bsmallmatrix}$, which is $\mathbf{W} = \mathbf{V}_d$, the first d columns of $\mathbf{V}$.

Any other allocation moves weight from a larger σ_j^2 to a smaller one, so the maximum is attained and no other subspace can beat it.

Step 7 · the error that is left

Substituting into step 2, and using $\|\tilde{\mathbf{X}}\|_F^2 = \mathrm{tr}(\mathbf{C}) = \sum_j \sigma_j^2$:

$$\min_{\mathbf{W}} E(\mathbf{W}) \;=\; \sum_{j} \sigma_j^2 \;-\; \sum_{j \le d} \sigma_j^2 \;=\; \sum_{j > d} \sigma_j^2$$

ECKART–YOUNG–MIRSKY, IN THE FROBENIUS NORM

✓ “PCA minimises reconstruction error” is a theorem, not a slogan — and the theorem also tells you the price in advance: the tail of the spectrum, which you can read off before choosing d.

The derivation, in seven lines

Reproduce this from the statement of the problem.

EXAMINABLE

1. $E(\mathbf{W}) = \|\tilde{\mathbf{X}} - \tilde{\mathbf{X}}\mathbf{W}\mathbf{W}^\intercal\|_F^2$
   the objective: squared distance to the subspace

2. $E(\mathbf{W}) = \|\tilde{\mathbf{X}}\|_F^2 - \|\tilde{\mathbf{X}}\mathbf{W}\|_F^2$
   Pythagoras; the first term is free of $\mathbf{W}$

3. $\|\tilde{\mathbf{X}}\mathbf{W}\|_F^2 = \operatorname{tr}(\mathbf{W}^\intercal\mathbf{C}\mathbf{W})$
   the Frobenius norm as a trace, with $\mathbf{C} = \tilde{\mathbf{X}}^\intercal\tilde{\mathbf{X}}$

4. $\mathbf{C} = \mathbf{V}\mathbf{\Sigma}^2\mathbf{V}^\intercal$
   from the SVD $\tilde{\mathbf{X}} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^\intercal$

5. $\operatorname{tr}(\mathbf{W}^\intercal\mathbf{C}\mathbf{W}) = \sum_j \sigma_j^2 h_j$, with $0 \le h_j \le 1$ and $\sum_j h_j = d$
   write $\mathbf{W} = \mathbf{V}\mathbf{A}$; h_j is a squared row norm of $\mathbf{A}$

6. the maximum is $\sum_{j \le d} \sigma_j^2$, at $\mathbf{W} = \mathbf{V}_d$
   spend all d units of weight on the largest σ_j^2

7. $\min_{\mathbf{W}} E(\mathbf{W}) = \sum_{j > d} \sigma_j^2$
   the error PCA leaves is exactly the tail of the spectrum

A theorem you can check to machine precision

```
U, S, Vt = np.linalg.svd(X - X.mean(axis=0), full_matrices=False)

d   = 100
Xd  = U[:, :d] * S[:d] @ Vt[:d]          # the rank-d truncation
lhs = ((X_centred - Xd) ** 2).sum()      # ‖X̃ - X̃_d‖²_F
rhs = (S[d:] ** 2).sum()                 # Σ_{j>d} σ_j²

assert abs(lhs - rhs) / rhs < 1e-5
```

Two sides computed by entirely different routes, agreeing to eleven significant figures. When a theorem hands you an identity, assert the identity: it is the cheapest possible test that your code is the theorem.

In scikit-learn the projection and the re-embedding are `transform` and `inverse_transform`. The second is not an inverse; it is the best re-embedding of a projection, and step 7 is what “best” means there.

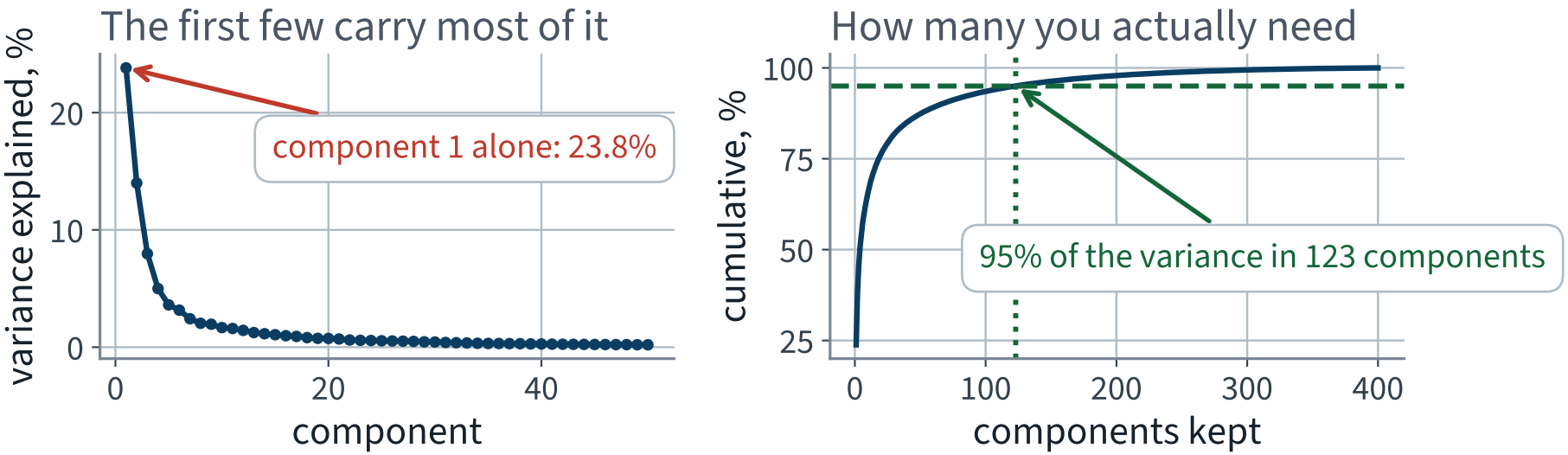
Choosing d without guessing: the explained variance ratio

$$r_j = \frac{\sigma_j^2}{\sum_i \sigma_i^2} \qquad \text{cumulate, and read off } d$$

By step 7, the fraction of squared error *removed* by keeping d components is exactly $\sum_{j \le d} r_j$. The ratio is not a heuristic; it is the theorem, normalised.

Cumulative variance explained by the first…	
component alone	23.8%
ten components	65.6%
fifty components	87.4%

How many components you actually need



123 components hold 95% of the variance of 4,096 pixels — a factor of 33 fewer numbers per photograph.

The second question: what if you only need *distances*?

PCA looks at the data and spends an SVD finding the best subspace. The alternative is to **not look at the data at all**: draw a random $\mathbf{R} \in \mathbb{R}^{n \times d}$ with independent Gaussian entries and set

$$\mathbf{Z} = \frac{1}{\sqrt{d}} \mathbf{X}\mathbf{R}$$

This sounds absurd. It is not, and the reason is a theorem — one that says nothing whatever about the subspace being good.

The Johnson–Lindenstrauss lemma

For any $0 < \varepsilon < 1$ and any set of m points in $\mathbb{R}^n$, there is a linear map f into $\mathbb{R}^d$ with

$$d \geq \frac{4 \log m}{\varepsilon^2/2 - \varepsilon^3/3}$$

such that for every pair of points $\mathbf{u}, \mathbf{v}$,

$$(1-\varepsilon)\|\mathbf{u}-\mathbf{v}\|^2 \leq \|f(\mathbf{u})-f(\mathbf{v})\|^2 \leq (1+\varepsilon)\|\mathbf{u}-\mathbf{v}\|^2$$

The tolerance is on **squared** distances. Quoting ε against unsquared ones understates the distortion by about a factor of two.

Read the formula again. What is missing?

$$d \geq \frac{4 \log m}{\varepsilon^2/2 - \varepsilon^3/3}$$

In the formula

- m, the number of points — logarithmically
- ε, the tolerance

Not in the formula

✗ n

The dimension you are starting from.

✓ 400 points need the same target dimension whether they live in 4,096 dimensions or in four million.

The bound, evaluated

```
from sklearn.random_projection import johnson_lindenstrauss_min_dim
johnson_lindenstrauss_min_dim(400, eps=0.2)          # 1382
```

ε	400 points	One million points
0.1	✗ 5,135	11,841
0.2	1,382	3,188
0.3	665	1,535
0.5	287	663

Two and a half thousand times as many points costs about *twice* the dimension. That is the $\log m$, and it is why the theorem is interesting at all.

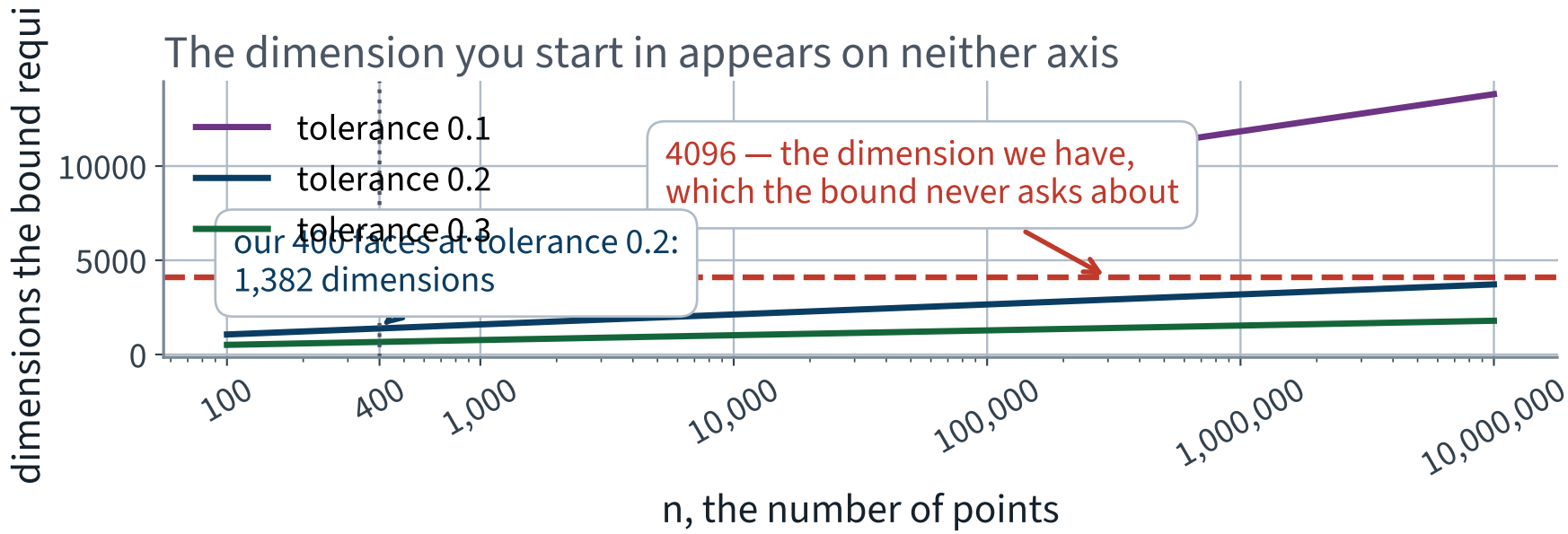
And be honest about the top-left cell

5,135

dimensions, to hold every pairwise distance among 400 faces to within 10% — which is **more** than the 4,096 we started with.

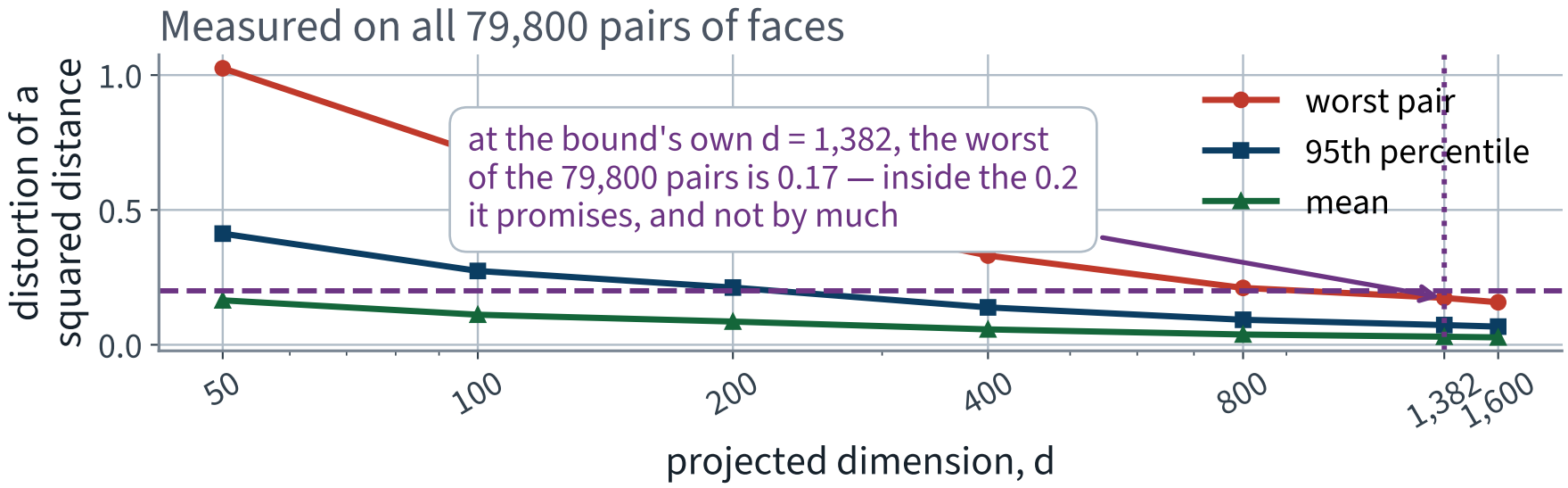
> **THE BOUND IS A WORST CASE OVER ALL POINT SETS**
>
> It has to work for the nastiest configuration of 400 points anyone can construct. A photographic archive is not that. So the bound is *sufficient*, never *necessary* — and the reason to teach it is not the constant.

The bound, drawn



Neither axis is the original dimension. The dashed line at 4,096 is drawn only to show that the bound never asks about it.

So measure what actually happens to our distances



At the bound's own $d = 1{,}382$ the worst of the 79,800 pairs is distorted by 0.17 — inside the 0.2 it promises, and not by much. The bound is loose, and not absurdly so.

What the mathematics buys you

- PCA is the SVD of the **centred** data; the components are the eigenvectors of $\tilde{\mathbf{X}}^\top \tilde{\mathbf{X}}$ with eigenvalues σ_j^2
- The principal subspace minimises squared reconstruction error exactly, and the error it leaves is $\sum_{j>d} \sigma_j^2$
- Minimising error and maximising projected variance are the same problem, by Pythagoras
- The explained variance ratio therefore chooses d against a stated target rather than by eye
- Distances can be preserved by a map whose target dimension depends on the number of points and the tolerance, and **not** on the dimension you start in

The last point returns in Lecture 23, where the same concentration argument explains why unrelated embeddings on a high-dimensional sphere are very nearly orthogonal.

THE METHOD, PART ONE

Reducing the dimension

25 minutes · examinable

The theorem, in one line of code

```python
from sklearn.decomposition import PCA

Z = PCA(n_components=0.95, random_state=42).fit_transform(X)
print(Z.shape)                                  # (400, 123)
```

`n_components=0.95` — a float between 0 and 1 is read as a **variance target** rather than a component count, and scikit-learn chooses d for you from the cumulative ratio. Here it chooses 123.

✓ An integer means “this many components”; a float means “this much variance”. One character apart, and `n_components=1` and `n_components=1.0` do completely different things.

The components live in the same space as the data

A principal component is a vector in $\mathbb{R}^{4096}$. Reshape it to 64 × 64 and it is an image.

> **WHY THEY HAVE A NAME**
>
> The principal components of a face dataset are called **eigenfaces**, because they are the eigenvectors of $\tilde{\mathbf{X}}^\top \tilde{\mathbf{X}}$ and they look like faces. Every photograph in the corpus is the mean face plus a weighted sum of them.

This is not a property of faces. It is a property of PCA: the components are directions in the input space, so whatever an input looks like, a component looks like too.

The mean face, then the first fifteen components

The mean face, then the first 15 principal components. Each component is itself an image

✗ Look at the first few. They are lighting — which is exactly what the distance histogram predicted would dominate.

The same held-out face, with components taken away

One held-out face, rebuilt from this many components

Recognisable well before 123 components. Identity survives a compression that the pixel values do not — and each PCA here was fitted on the training faces only.

Four ways to reduce a dimension

Method	What it does	When
`PCA(svd_solver="full")`	the complete SVD, then truncate	small matrices; exact
`PCA(svd_solver="randomized")`	approximates the top d components	$d \ll \min(m, n)$
`IncrementalPCA`	one batch at a time, never holds $\mathbf{X}$	data larger than memory
`GaussianRandomProjection`	draws $\mathbf{R}$; never looks at the data	n enormous, reconstruction not needed

The default `svd_solver="auto"` chooses between the first two by a size rule. That is a default you did not ask for; print `pca.svd_solver_` if a timing surprises you.

Timed and scored, rather than asserted

All four at $d = 123$, fitted on the 280 training faces, reconstruction error measured on the 120 held out. Times are the best of three on one machine at two threads; the *error* column is the one that reproduces anywhere.

Method	Fit time, s	Held-out error
PCA, full SVD	0.074	0.00240
PCA, randomised	0.081	0.00241
Incremental PCA, 3 batches	0.142	0.00240
Random projection	**✓ 0.006**	**✗ 0.32093**

Three PCA variants land on the same error at different prices. Random projection is essentially free and pays for it with a reconstruction error **133 times** larger — a random 123-dimensional subspace of $\mathbb{R}^{4096}$ keeps only about $123/4096 = 3.0\%$ of a generic vector's squared norm.

✓ That is not a bug in random projection. PCA minimises exactly this column by construction; Johnson–Lindenstrauss promises something else entirely, that *relative distances* survive. Neither theorem is about the other quantity.

Choosing between them, as a rule

USE PCA WHEN

you will reconstruct, visualise or interpret the components; when the data fits in memory; when d should be chosen against a variance target.

USE RANDOM PROJECTION WHEN

the ambient dimension is so large that the SVD itself is the bottleneck, and all you need is that distances survive.

So the question is never which is better. It is whether you need the **reconstruction** or only the **distances**.

✓ Here, PCA. 400 × 4,096 is a small matrix, and the components are interpretable images.

When PCA is the wrong tool

THREE CONDITIONS, EACH OF WHICH BITES HERE OR NEARBY

1. Variance is not signal. PCA keeps the directions of largest variance. If the largest variance is a nuisance — a lamp — PCA keeps the nuisance first.

2. The structure is not linear. A subspace is flat. Data on a curved manifold needs kernel PCA, LLE or a learned embedding.

3. The features are not commensurable. PCA is not scale-free: change the units of one column and the components change. Standardise first unless, as here, every feature is already the same kind of number.

Condition 1 is the one on this data, and no amount of compression repairs it — measured at the end of the lecture.

One procedural condition, because it is invisible

A reducer is an estimator: it is **fitted**. Fit it before the split and the held-out faces have helped choose the subspace they are then judged in.

```
pca = PCA(n_components=0.95).fit(X)          # all 400 photographs
Z   = pca.transform(X)

Z_tr, Z_te, y_tr, y_te = train_test_split(   # the split comes after
    Z, y, test_size=0.3, stratify=y, random_state=42)
```

The 280 training faces span at most a 280-dimensional subspace of $\mathbb{R}^{4096}$, so adding 120 more points genuinely changes which directions survive. This is not a rounding error.

What it costs, over twenty splits

$d = 123$, 20 stratified splits	Fitted on train	Fitted on everything	Leaky wins
Reconstruction error, held out	0.00243	0.00096	✗ 20 of 20
Accuracy, held out	97.4%	97.4%	3 of 20

The leak makes the reconstruction error look **61%** better and moves the accuracy by nothing measurable.

THE DECISION RULE, AND WHY IT IS STILL PROCEDURAL

Fitting an unsupervised step on everything costs you **exactly when that step's own objective is the number you report**. Reconstruction error is that objective; downstream accuracy is not — and you cannot tell which row you are in without doing the split. So: `Pipeline([("pca", PCA(0.95)), ("clf", …)])`, which refits the PCA inside every fold and makes the leak structurally impossible rather than merely avoided.

THE METHOD, PART TWO

Clustering, and how to choose k

20 minutes · examinable

k-means, in five lines

1. Choose k centroids (scikit-learn uses *k-means++*, which spreads the initial ones out)
2. Assign every point to its nearest centroid
3. Move every centroid to the mean of its assigned points
4. Repeat 2–3 until no assignment changes
5. Repeat the whole thing `n_init` times and keep the best

> Step 5 is not optional. Lloyd's algorithm converges — to a local minimum that depends entirely on where you started. `n_init` changed its default between scikit-learn versions, so name it.

What k-means assumes, whether you asked or not

FOUR ASSUMPTIONS, THREE OF THEM FALSE HERE

Convexity. The partition is a Voronoi diagram of the centroids, so every cluster is convex.

Spherical, equal-sized. The only quantity is distance to a centre.

Total assignment. Every point belongs to exactly one cluster; there is no "none of these".

k is given. Nothing in the algorithm chooses it.

The first three are dropped by DBSCAN and by Gaussian mixtures, in the last block of this lecture. The fourth — choosing k — is what the rest of this section is about.

Every metric you know needs labels

Metric	Needs	Available here?
RMSE	a true value per instance	**no ✗**
Accuracy, F_1, ROC AUC	a true class per instance	**no ✗**
Inertia	nothing	**yes ✓**
Silhouette	nothing	**yes ✓**
Adjusted Rand index	labels — a subset is enough	**on the forty ✓**

Two of these can *choose* a model. One of them can be *shown to a stakeholder*. They are not the same one, and keeping those two jobs apart is the standing rule of this course applied where there is no test set at all.

Inertia, and why it cannot choose k

$$J(k) = \sum_{i=1}^{m} \left\| \mathbf{x}^{(i)} - \boldsymbol{\mu}_{c(i)} \right\|_2^2$$

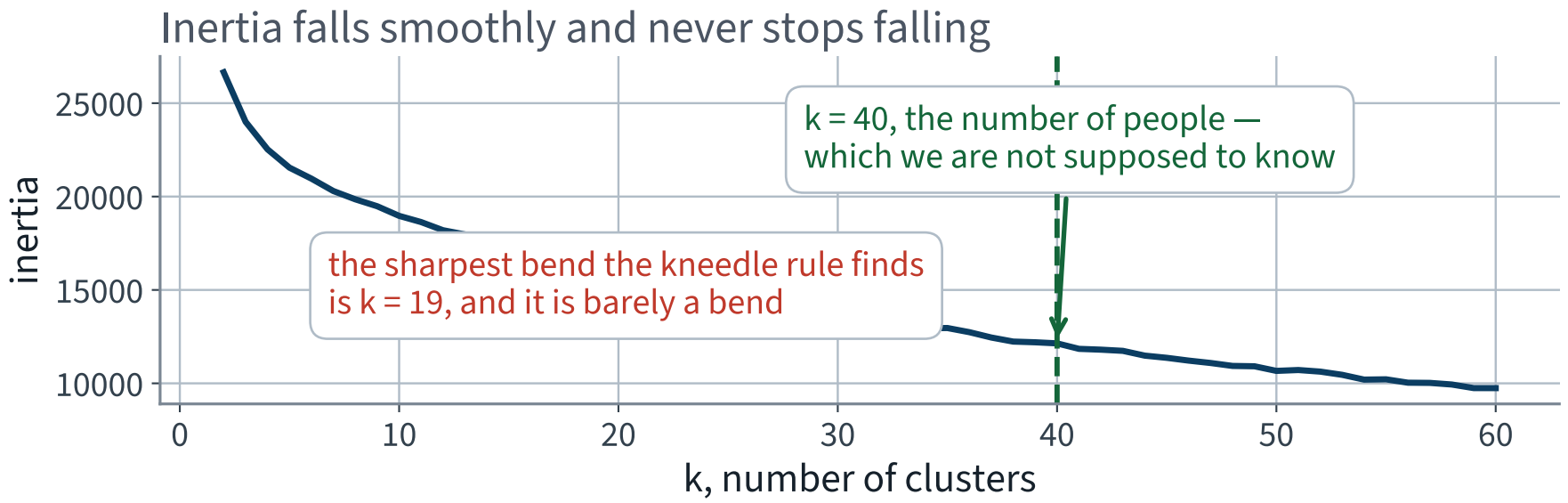
THE SUM OF SQUARED DISTANCES TO THE ASSIGNED CENTROID — WHAT K-MEANS MINIMISES

A PROPERTY, NOT AN ACCIDENT

J is **non-increasing in k** — refining a partition cannot increase the sum of squared distances to the assigned centre — and $J(m) = 0$ exactly. There is no interior minimum to find, at any k, on any dataset.

So “minimise inertia” selects one cluster per photograph. The elbow rule exists to work around that.

Find the elbow



A smooth convex curve. The *kneedle* rule — furthest below the chord, both axes rescaled — picks $k = 19$; the archive has 40, and 46% of the inertia at $k = 2$ is still present at $k = 40$.

A criterion that *does* have an interior maximum

✗ The elbow is not a criterion. It is a picture of a criterion that does not exist, and no better heuristic repairs it, because the information is not in J. So use one that does.

For a point i, let a be its mean distance to the other points of *its own* cluster, and b its mean distance to the points of the *nearest other* cluster.

$$s_i = \frac{b - a}{\max(a, b)} \in [-1, 1]$$

- $s_i \to +1$: far inside its own cluster · $s_i \approx 0$: on a boundary · $s_i \to -1$: assigned to the *wrong* cluster
- The **silhouette score** is the mean of s_i. It is penalised for splitting a group *and* for merging two, so unlike inertia it has an interior maximum
- It costs $O(m^2 n)$ — every pair, over every feature. Both factors matter: the archive is bigger than 400, and n is larger than it needs to be

Before reading any silhouette: what does *nothing* score?

Put every photograph in a uniformly random group, twenty times, and look at the spread. Both metrics have no natural scale, and a number with no null has no units.

Random assignment	Silhouette	ARI on the forty
$k = 10$	−0.0422 ± 0.0045	−0.0040 ± 0.0205
$k = 40$	−0.1288 ± 0.0107	−0.0089 ± 0.0244
$k = 60$	−0.1805 ± 0.0124	−0.0075 ± 0.0261

✓ Both anchors sit at zero, and now you know the width of the noise around zero. A silhouette of 0.02 is not a discovery.

The number the stakeholder reads: adjusted Rand index

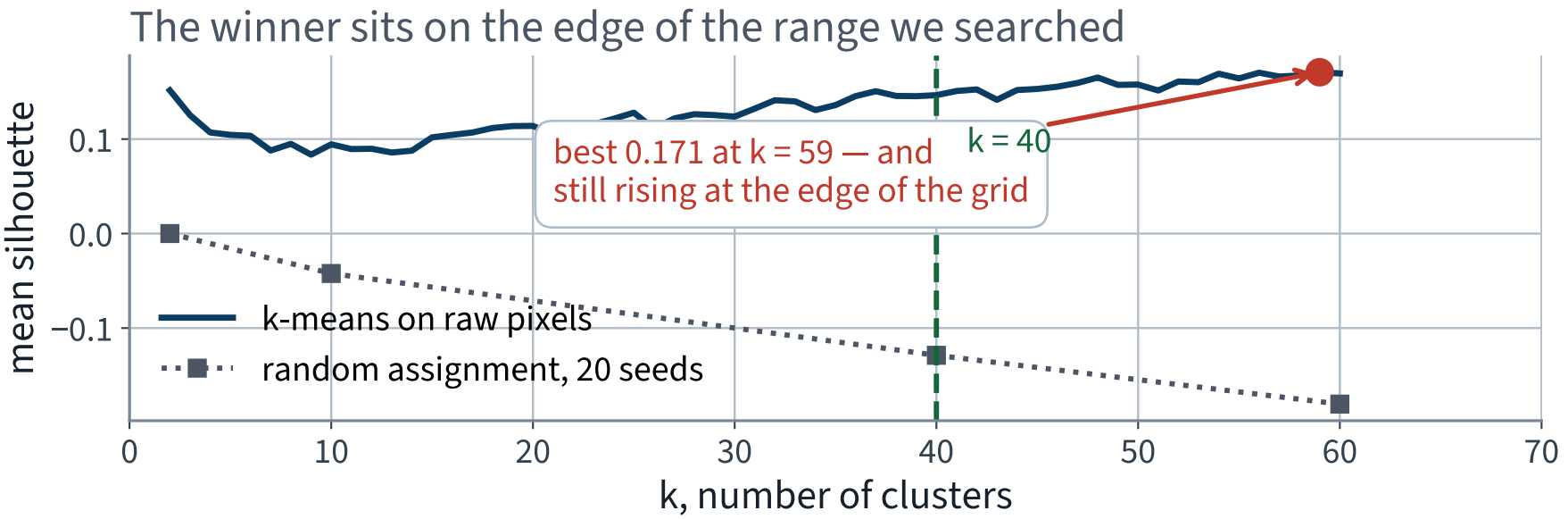
Compare two groupings pair by pair, and count the pairs they agree about — together in both, or apart in both:

$$\mathrm{RI} = \frac{a + b}{\binom{m}{2}}, \qquad \mathrm{ARI} = \frac{\mathrm{RI} - \mathbb{E}[\mathrm{RI}]}{1 - \mathbb{E}[\mathrm{RI}]}$$

RI is high for free at large k — almost every pair is apart in both groupings. The *adjusted* index subtracts what chance would give, so random scores 0 in expectation at any k, perfect scores 1, and negative is worse than chance.

✗ We can compute it on the forty labelled photographs only: 780 pairs, of which just 22 are the same person. Treat it as a sanity check with a wide interval, never as a ranking.

Silhouette against k, read against the anchor



A maximum exists. It is also low, and the curve near it is nearly flat — so the chosen k is not decisively better than its neighbours.

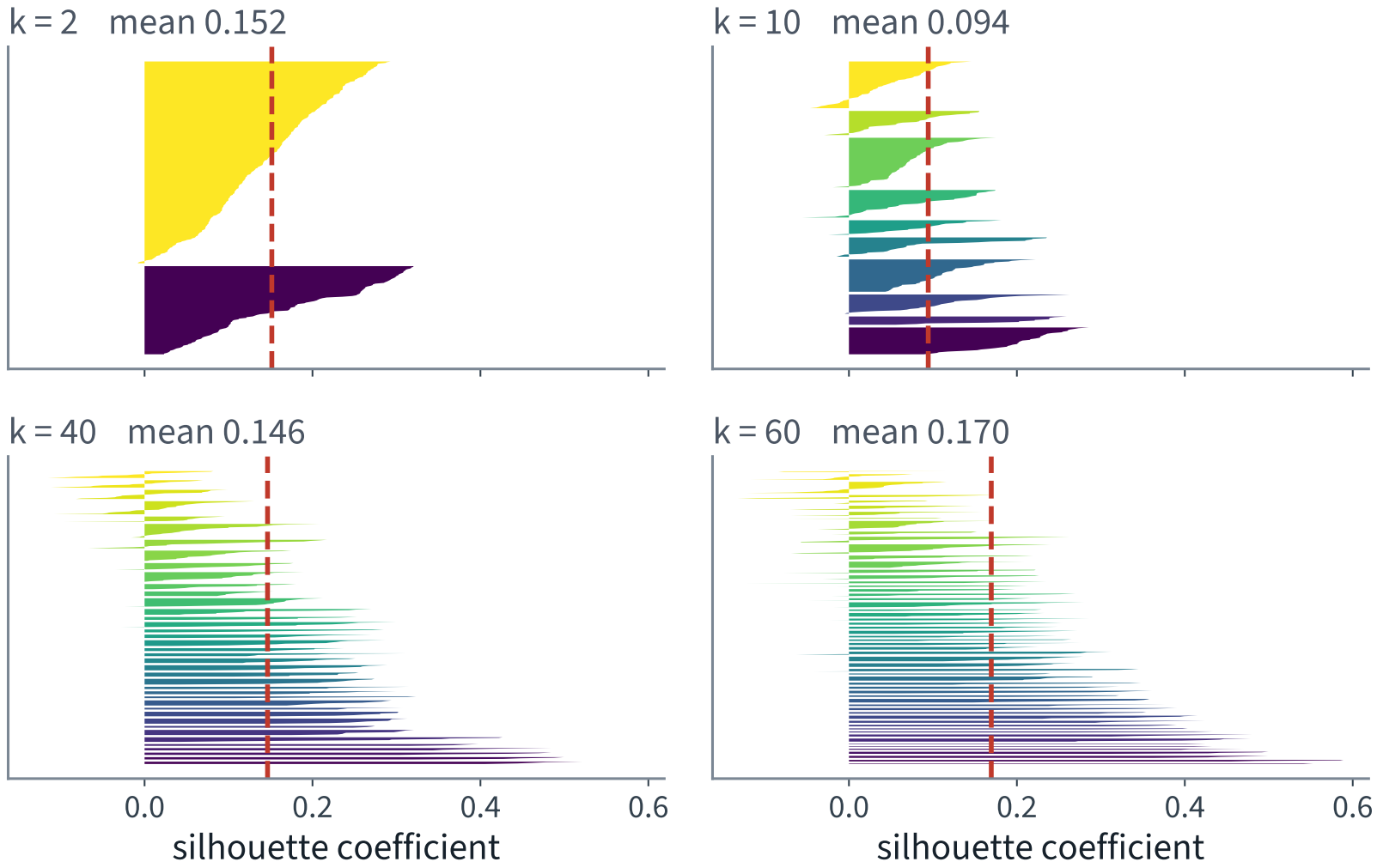
What it chose

	k	Silhouette	ARI on the forty
Random assignment	40	−0.1288	−0.0089
The silhouette's choice	59	**0.1707 ✓**	0.3954
The true k — an audit	40	0.1465	0.2978

Well above the anchor, so the clustering is doing something. It chose 59 groups for 40 people, and the silhouette at the truth is *lower* than at its own choice.

The last row is an audit: on the slide because you should see it, not because the project could have computed it.

The mean hides the shape



One knife per cluster: width is size, length is quality. At larger k they are short and uneven.

How to read a silhouette diagram

What you see	What it means
A knife entirely left of the dashed mean	that cluster is worse than average; consider a different k
Knives of very unequal width	the sizes are unbalanced — one group has swallowed several
A knife extending into negative s_i	those points are closer to another cluster than to their own
Every knife short	the clusters overlap; the geometry has no clean gaps

✗ Ours is the last row, at every k we tried. That is the diagnosis the mean silhouette of 0.1707 does not contain.

A silhouette of 0.1707 means nothing until you look

cleanest cluster at k = 40: 10 photographs, 1 identity, purity 1.00

worst cluster at k = 40: 24 photographs, 10 identities, purity 0.21

✗ **The cleanest and the worst cluster at $k = 40$. The bottom row is not a person. It is a *lighting condition* — several different people lit and posed the same way.**

Counting it, rather than describing it

At $k = 40$, against the hidden identities	Value
Cleanest cluster: 10 photographs, purity	**1.00 ✓**
Worst cluster: 24 photographs of 10 people, purity	**✗ 0.21**
Mean purity across the forty clusters	0.74
Clusters holding exactly one person	16
Clusters with exactly ten members	7
Cluster sizes, smallest to largest	4 to 25

Every true group has exactly ten members. The failure condition stated at the start of the lecture is now a measurement.

A last property of choosing k by a score

THE MAXIMUM OF SEVERAL NOISY ESTIMATES IS BIASED UPWARDS

Sweep k, keep the best silhouette, and print it, and the number that *chose* the model is also the number that *reports* it. The bias grows with how many candidates you tried, and it does not need labels to exist.

Measured: choose k on a random half of the corpus, score the chosen model on the other half, twenty times.

Silhouette, 20 random halves	Mean	sd
Reported — chose and scored on the same half	0.1777	0.0123
Held out — had no vote	0.0878	0.0154
Optimism	✗ 0.0899	0.0229

✗ Half the reported score, and worse in 20 splits out of 20. Report the held-out number, or report both.

Now put the two halves of the lecture together

```
Z = PCA(n_components=0.95, random_state=42).fit_transform(X)   # 123

for k in range(2, 61):
    km = KMeans(n_clusters=k, n_init=10, random_state=42).fit(Z)
    sil.append(silhouette_score(Z, km.labels_))
```

The clustering code is unchanged. The grid is unchanged, the seed is unchanged, `n_init` is unchanged. Only *what k-means is looking at* changed.

✓ One line, and both cost models lose a factor of 33 in their n.

The trade-off, measured rather than asserted

Dimensions	Sweep, seconds	Best k	Silhouette	ARI, all 400
4,096 — raw pixels	✗ 78	59	0.171	0.537
20	1.9	58	0.286	0.540
50	1.7	59	0.256	0.493
100	1.9	60	0.218	0.547
123 — 95% of the variance	✓ 2.1	60	0.205	0.524
399	5.8	59	0.171	0.537

The sweep is **37 times** faster at 123 dimensions, and the agreement with the true identities does not fall. The ARI column is there because a speed-up that quietly costs quality is not a speed-up.

Every second in that column is a property of one machine at two threads, and so is the ratio: run the notebook and you will measure a different speed-up, because the fixed costs do not shrink with n. What is a property of the *method* is the shape — an order-of-magnitude saving, bought at no cost in ARI.

Careful: the silhouette went *up*

> **THIS IS NOT EVIDENCE THAT THE CLUSTERING IMPROVED**
>
> The silhouette is computed in whatever space you hand it. Compressing the data changes every distance, so the criterion is **not comparable** across the rows of that table.

Which is exactly why the ARI column is there. It is measured against the identities and does not depend on the representation, so it is the only column whose rows may be compared.

The general rule: never compare two scores measured on different evaluation sets — and a different representation is a different evaluation set.

FURTHER GROUND

Density, mixtures, anomalies, forty labels

15 minutes · examinable

Compression fixed the clock. It did not fix the geometry

Every k-means cluster is still a convex Voronoi cell, every point still belongs to exactly one, and every group is still assumed to be the same size. Two methods from Chapter 8 that drop those assumptions:

	DBSCAN	Gaussian mixture
Cluster shape	any connected dense region	an ellipsoid
Number of clusters	found, not given	given
Every point assigned?	no — there is a noise label	softly, with a probability
Main parameter	`eps`, a distance	`n_components`

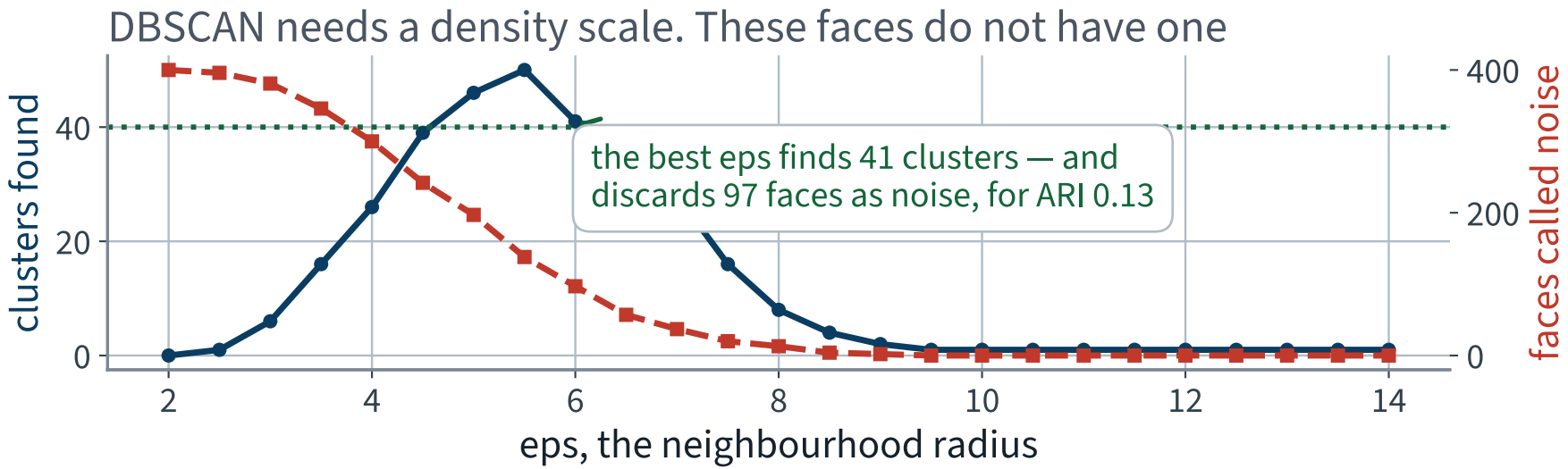
DBSCAN, in three definitions

- A point is a **core point** if at least `min_samples` points lie within `eps` of it
- Core points within `eps` of each other are in the same cluster; so are the non-core points in their neighbourhoods
- Everything else is **noise**, labelled -1

```
db = DBSCAN(eps=6.0, min_samples=3).fit(Z)
n_clusters = len(set(db.labels_) - {-1})     # exclude the noise label
```

`DBSCAN` has no `predict`. It is transductive: to label a new point you fit again, or you train a classifier on its output.

DBSCAN on the compressed faces



At small `eps` everything is noise; at large `eps` everything is one cluster. The most it ever finds is 50, and never 40.

Reporting a method that did not work

DBSCAN NEEDS ONE DENSITY SCALE THAT SEPARATES CLUSTERS FROM THE GAPS

Best over the whole grid: ARI **0.133** at `eps` = 6.0, with 41 clusters and 97 of the 400 faces called noise. Forty groups of ten faces, all lit by the same rig, have no such scale: the gap between two people is about the gap between two photographs of one person.

✓ This is a real result and it belongs on the slide. A method that fails for a stateable reason is more informative than a method that was never tried.

Gaussian mixtures

Model the data as coming from k Gaussians with unknown means, covariances and weights, fitted by expectation–maximisation:

$$p(\mathbf{x}) = \sum_{j=1}^{k} \pi_j \, \mathcal{N}\left(\mathbf{x};\, \boldsymbol{\mu}_j,\, \boldsymbol{\Sigma}_j\right)$$

- Clusters are ellipsoids, so they may be elongated and of different sizes
- Assignment is *soft*: `predict_proba` returns a distribution over components
- It is a **density model**, so it also scores how likely a new point is — which is anomaly detection, shortly

Why this needed the first half of the lecture

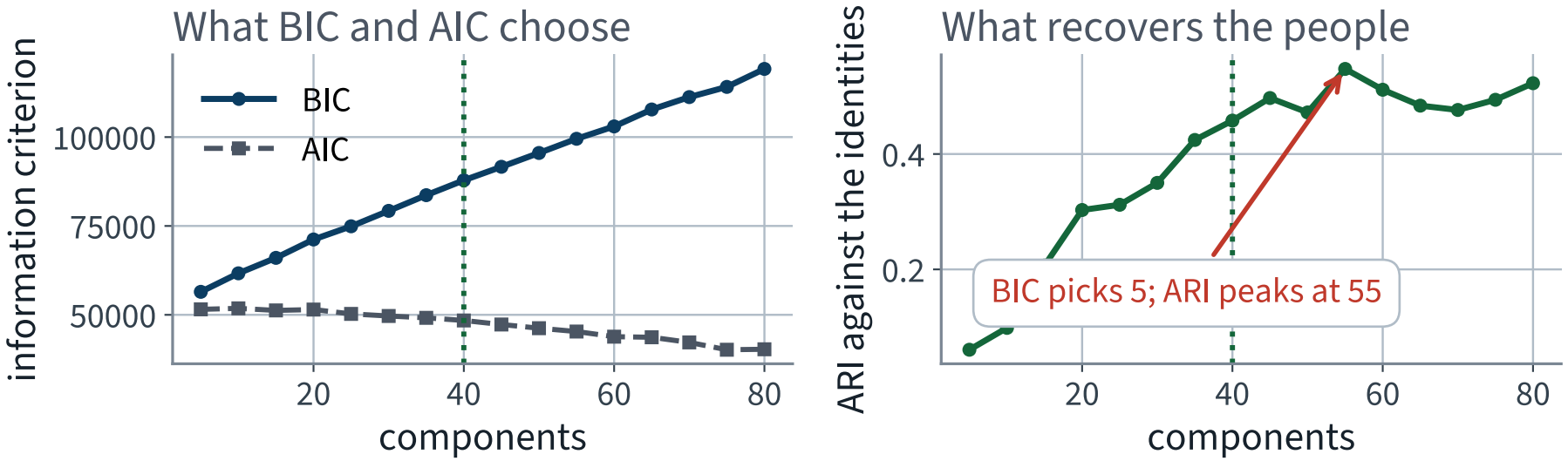
A full covariance in n dimensions has $n(n+1)/2$ free parameters.

Setting	Free parameters
One full covariance in 4,096 dimensions	8,390,656
Forty of them	✗ 335,626,240
Estimated from	400 photographs
Forty *diagonal* components in 123 dimensions — means, variances and weights, the whole model	✓ 9,880

✗ The first is not slow. It is undefined: every covariance estimate is singular. This is a question arithmetic settles in one line, and an afternoon of waiting does not.

`covariance_type` is `"full"`, `"tied"`, `"diag"` or `"spherical"`. We use `"diag"`, and that is a decision, not a default.

What BIC chooses, and what recovers the people



BIC picks 5 components; the ARI peaks at 55. The criterion you can compute and the answer you want are not the same function.

BIC and AIC, and what they are for

$$\mathrm{BIC} = p \log m - 2 \log \hat{L}, \qquad \mathrm{AIC} = 2p - 2 \log \hat{L}$$

p is the number of parameters, m the number of instances, $\hat{L}$ the maximised likelihood. Both penalise complexity; BIC penalises it harder once $\log m > 2$, so it tends to choose fewer components.

THEY ANSWER A DIFFERENT QUESTION

BIC asks which model best explains this data as a *density*. We are asking which grouping matches the people. Those coincide only when the people really are Gaussian blobs, and here they are not.

Anomaly detection, two ways

By density

A Gaussian mixture is a density. Score every image and call the lowest few anomalous.

```
d   = gmm.score_samples(Z)
bad = d < np.percentile(d, 3)
```

By reconstruction error

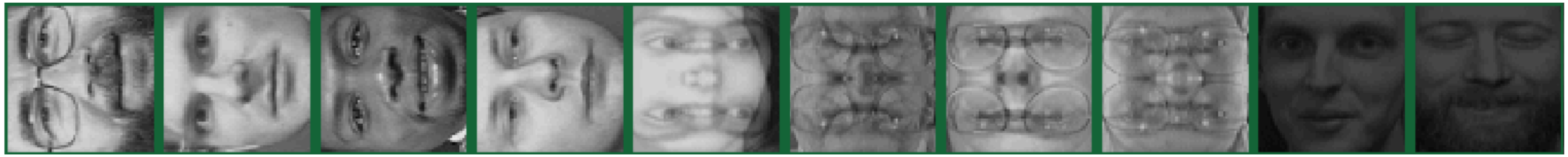
An image the eigenfaces cannot rebuild is unlike the images that built them.

```
R   = pca.inverse_transform(pca.transform(Xa)
err = ((R - Xa) ** 2).mean(axis=1)
```

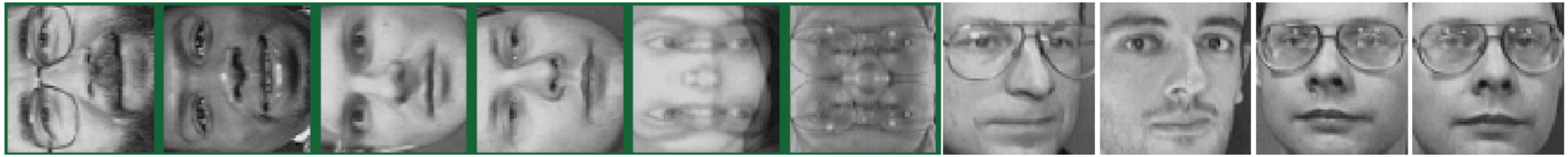
Test both by planting twelve corrupted images — four rotated, four dimmed, four double-exposed — and counting how many each puts in its top twelve.

What each detector found

the ten most anomalous by reconstruction error — 10 of these ten are planted (12 of 12 caught in the top twelve)



the ten most anomalous by lowest mixture density — 6 of these ten are planted (6 of 12 caught in the top twelve)



Reconstruction error caught 12 of the 12 planted images; the mixture density caught 6.

Why one of them won

- A rotated or double-exposed face is **outside the subspace** the eigenfaces span, so its reconstruction error is large — median 0.0017 against 0.0002 for a genuine face
- The mixture density is estimated *inside* that subspace, so a corruption that projects onto an ordinary-looking point is invisible to it

> **THE DECISION RULE**
>
> Use the **reconstruction error** when the anomaly is a different *kind* of object. Use the **density** when it is an ordinary object in an unusual place.

Finally: the forty labels

Split the corpus 280 / 120, stratified by identity. We may label forty of the 280 training photographs. Where we spend them turns out to matter more than what we do with them.

1. Forty **at random**
2. Forty chosen as the photograph nearest each of forty cluster centroids — `km.transform(Z).argmin(axis=0)`
3. The representative's label **propagated** to its whole cluster — 280 labels from 40
4. Propagated only to the 75% of each cluster closest to its centroid
5. All 280 true labels — the ceiling, which the project cannot buy

Same classifier, same held-out faces, twenty splits. The only thing that varies is *which* forty were labelled.

The numbers

Labels used	Held-out accuracy	sd over 20 splits
40 at random	48.1%	3.8%
40, one per cluster	**60.5% ✓**	4.9%
Propagated to the whole cluster	55.7%	4.7%
Propagated to the closest 75%	55.1%	4.8%
All 280 true labels	**✓ 97.4%**	1.6%

Twenty splits, because the differences between the middle three rows are of the same order as their spread — and a comparison that turns on less than the spread is not a comparison.

Propagation buys 280 labels from 40 and does *worse* than the 40 alone, because a wrong propagated label is worse than no label. Report that, do not bury it.

The diagnosis was right. The obvious repair still fails

Lighting dominates the distance. Two one-line repairs, on the same 400 faces, choosing k by silhouette and reporting ARI exactly as before:

Representation	Best k	Silhouette	ARI, all 400
PCA to 123 dimensions	60	0.205	**0.524 ✓**
PCA 123, first 3 components discarded	60	0.184	**✗ 0.508**
Unit-norm each image, then PCA 123	58	0.189	**✗ 0.464**

Neither helps. The leading components carry pose and face shape as well as illumination, so removing them throws away signal with the nuisance. The remedy that does work is a representation learned for the task rather than for variance — which is what Part VI builds, and why a pretrained encoder beats a hand-chosen projection.

✗ Note also what chose k. Sweeping k and keeping the best *ARI* makes both repairs look like gains — because ARI is computed from the labels this whole lecture is pretending not to have.

Specimen exam question

> A team has 5,000 documents represented as 50,000-dimensional vectors. They want to preserve pairwise distances to within 20% while reducing the dimension.

1. Using the Johnson–Lindenstrauss bound, state the target dimension — and say which of the two given numbers you did not need
2. They acquire ten times as many documents. By roughly what factor does the required dimension grow?
3. Give one reason to prefer PCA here anyway, and one reason not to

Part 1 is two marks: one for the number, one for saying that 50,000 does not enter it.

Specimen answer

1. $d \geq 4\log(5000)/(0.02 - 0.00267) \approx 1{,}965$. The **50,000 is not used**: the bound contains only the number of points and the tolerance
2. $\log(50{,}000)/\log(5{,}000) \approx 1.27$, so about a **27% increase** — not ten times
3. For PCA: the components are interpretable, and 5,000 × 50,000 is still tractable, and PCA is far better if anything downstream needs the documents reconstructed. Against: it must look at all the data and costs an SVD, and it gives no distance guarantee at all

✗ The common wrong answer to 2 is "ten times". The bound is logarithmic in the number of points, and that is the only reason it is a useful theorem rather than a curiosity.

What is examinable from today

- The derivation of the principal subspace: the objective, Pythagoras, the trace, the SVD, the weighted-sum maximum, and $\min E = \sum_{j>d} \sigma_j^2$
- The explained variance ratio, and choosing d from a variance target
- The Johnson–Lindenstrauss bound: stating it, evaluating it, and saying what is *not* in it
- k-means and its four assumptions; why inertia cannot choose k; the silhouette and its cost; reading a silhouette diagram
- ARI as a chance-corrected agreement between two groupings
- DBSCAN and Gaussian mixtures: what each assumes and when each fails; BIC and AIC; anomaly detection by density and by reconstruction error; label propagation from cluster representatives

Colab mechanics, thread pinning, absolute wall-clock timings, and the internals of `fetch_olivetti_faces`. NOT EXAMINABLE

Also in Chapters 7 and 8 and cut for time: kernel PCA and locally linear embedding; t-SNE and UMAP, which are visualisations rather than feature maps; Bayesian Gaussian mixtures, which drive unneeded components' weights to zero and so choose k themselves; and Chapter 8's survey of agglomerative, mean-shift, spectral and BIRCH clustering. Named because a method you did not try is not a method that failed. FOR CONTEXT

The notebook for this lecture

Open Lecture 8 in Colab

- **Runs on free CPU**, top to bottom, in three to five minutes on Colab. The slowest cell is the 4,096-dimensional sweep, and it says so
- It verifies the derivation numerically: the components agree with scikit-learn to 10^{-4}, and Eckart–Young closes to 10^{-5} relative
- Everything on today's slides — the distance histogram, both sweeps, the silhouette diagrams, the four reducers, DBSCAN, the mixture, the anomalies, the forty labels and the leak

> **WHAT TO CHANGE, AND WHAT SHOULD HAPPEN**
>
> Every prompt box carries a **try** line. The cheapest one: change `n_components=0.95` to `0.99`, which takes d from 123 to 260. The sweep slows by roughly the ratio of the two, and the ARI barely moves — which is the whole trade-off in one edit.

Every notebook in the course

01 · What machine learning is, and how we will work

02 · The end-to-end project

03 · Classification and its metrics

04 · Training models

05 · Regularisation and the bias–variance trade-off

06 · Decision trees

07 · Ensembles and random forests

08 · Dimensionality reduction and unsupervised learning

09 · Neural networks, from the perceptron up

10 · PyTorch

11 · Training deep networks

12 · Convolutional networks

13 · Transfer learning

14 · Detection and segmentation

15 · Time series

16 · Recurrent networks

17 · Text

18 · Attention and transformers

19 · Information retrieval: the lexical foundation

20 · Information retrieval: dense retrieval

21 · Recommender systems: from ratings to factors

22 · Recommender systems: neural, and evaluated honestly

23 · Vision transformers and multimodal retrieval

24 · Generation, retrieval-augmented systems, and where this leaves you

What to remember

1. PCA is the SVD of the **centred** data. The principal subspace minimises squared reconstruction error exactly, and the error it leaves is $\sum_{j>d} \sigma_j^2$ — the tail of the spectrum, readable before you choose d
2. Minimising that error and maximising the projected variance are the same problem, by Pythagoras. The explained variance ratio is that theorem normalised, and it is how d is chosen: 123 components hold 95% of 4,096 pixels
3. Johnson–Lindenstrauss bounds the target dimension by the *number of points* and the *tolerance*, never by the dimension you started in — and it is a loose worst case, so measure the distortion you actually get
4. Inertia cannot choose k: it is monotone in k and zero at $k = m$. The silhouette can, at $O(m^2 n)$, and must be read against a random anchor — and the score that chose the model is optimistic by about half, here
5. k-means assumes convex, equal-sized clusters and a known k. DBSCAN and Gaussian mixtures drop the first two, and on this data DBSCAN fails for a stateable reason: there is no single density scale
6. Reconstruction error finds anomalies of a different *kind*; density finds ordinary objects in unusual places
7. Forty labels spent on cluster representatives are worth a quarter more accuracy than forty spent at random. Which forty mattered more than what was done with them

In the book

Topic	Chapter
The curse of dimensionality; projection and manifolds	7
PCA, explained variance ratio, choosing the number of dimensions	7
Randomised PCA, incremental PCA	7
Random projection and the Johnson–Lindenstrauss bound	7
Locally linear embedding and other techniques	7
k-means, inertia, the silhouette; limits of k-means; DBSCAN	8
Gaussian mixtures, EM, BIC and AIC, anomaly detection	8
Clustering for semi-supervised learning and label propagation	8

NOT PRESENTED — FOR AFTERWARDS

Exercises Lecture 8

five, in the style of the written paper

Exercises — Lecture 8

1 PCA is derived by maximising $\|XW\|_F^2$ subject to $W^\top W = I$. State why the data must be centred first, and what the first component becomes if it is not. [5]

2 The Johnson–Lindenstrauss bound, evaluated for 400 images at $\varepsilon = 0.1$, asks for more dimensions than the data has. What do you conclude about the bound, and about the method? [5]

3 A pipeline fits PCA on the whole dataset and then splits. Name what leaked, and say whether the leak grows or shrinks as the corpus grows. [4]

Solutions on Lecture 9's deck.

Exercises — Lecture 8 (continued)

4 Inertia always falls as k rises, so it cannot choose k. State what silhouette adds, and one situation where it also fails. [5]

5 You have 4,096-dimensional face vectors and are told two faces are “close”. Explain why that alone does not mean they are the same person. [4]

Solutions on Lecture 9’s deck.

THE FIVE SET AT THE END OF LECTURE 7

Solutions Lecture 7

not part of this lecture

Solutions — Lecture 7

1. For n predictors each of variance σ^2 and pairwise correlation ρ, the variance of their average…

✓ It tends to $\rho\sigma^2$ — correlation, not ensemble size, sets the floor.

- The second term vanishes; the first does not depend on n at all
- Adding members past a point buys nothing, which is why the design effort goes into *decorrelating* them

2. Extra-trees are usually faster to fit than a random forest and often no less accurate. Give the mechanism, in…

✓ Speed, not a lower floor. Not searching for the best threshold is what makes them fast; measured against the forest their ρ is ***higher*** (0.067 against 0.043), and they stay competitive because the two mechanisms are substitutes rather than additions.

- Extra-trees without a bootstrap land at $\rho = 0.067$, near bagging; with one they land at 0.043, exactly where the forest is — random thresholds substitute for the bootstrap rather than adding to it
- Their individual members are *better*, not worse — 77.8% against 73.6% — because a random threshold on every feature beats an

Solutions — Lecture 7 (2)

3. You set bootstrap=False and oob_score=True. State what happens and why.

✓ It raises: with no bootstrap there are no out-of-bag rows.

- Out-of-bag scoring uses the rows a member did not sample
- Without sampling, every member sees every row and the estimate is undefined

4. Impurity-based feature importance ranks a random decoy column above several real ones. Explain how that can…

✓ A high-cardinality column offers more split points, so it accumulates impurity decrease by chance. Use permutation importance.

- Impurity importance rewards a column for being *splittable*, not for being informative
- Permutation importance measures the score lost when the column is shuffled, which a decoy cannot fake

Solutions — Lecture 7 (3)

5. A forest improves accuracy over one tree by far less than the variance reduction would suggest. Is this a…

✓ No — variance is only one term of the error.

- Averaging reduces variance and leaves bias untouched
- If the remaining error is mostly bias or noise, removing variance moves the accuracy very little

LECTURE 9 · GÉRON CH. 9

Neural networks, from the perceptron up

The perceptron, what it can and cannot separate
The multilayer perceptron, and why a stack of linear layers is a linear model
What a layer computes, in matrix form, with the shapes tracked

Run the Lecture 8 notebook first